

Softwaretechnische Änderungen zur Verwaltung der PCL-Daten

PCL-Visualizer Verbesserung an Zuverlässigkeit der Bildspeicherung sowie hinzufügen eines Auswahlbuttons

Ausgangslage:

Wollen hier Objekte gelöscht werden, welche in dem Visualizer platziert werden müssen sie mit dieser Logik gelöscht werden. Problem ist jedoch das es aus irgendeinem Grund nur einmal funktioniert, beim zweiten Löschen kann es sein das das Programm abstürzt weil er den Pointer oder die Adresse verliert.

Die Vermutung ist das es an den geteilten Ressourcen liegt, da die Visualisierung und die Verarbeitung der Punkte in einem Thread laufen.

GGF. Kann man versuchen mit `std::mutex` zu arbeiten, um die Ressourcen zu schützen. Dies wurde aufgrund von Zeitmangel noch nicht ausprobiert.

```
if (this->objectCount > 0)
{
    for (this->objectCount; this->objectCount > 0; this->objectCount--)
    {
        this->aPCLVisualizer->removeShape(std::to_string(this->objectCount));
    }
}
```

RemoveShape verursacht einen Fehler in der Engine beim ersten mal löschen, beim zweiten mal kann es sein das es entweder geht oder abstürzt -> Grund ist unklar.

```
this->aPCLVisualizer->removeCoordinateSystem(std::to_string(this->objectCount));
    }
    this->objectCount = 0;
}
```

Mögliche Methoden die dafür in Frage kommen:
(aus „pcl_visualizer“)

```
/** \brief Removes a Point Cloud from screen, based on a given ID.
 * \param[in] id the point cloud object id (i.e., given on \a addPointCloud)
 * \param[in] viewport view port from where the Point Cloud should be removed (default: all)
 * \return true if the point cloud is successfully removed and false if the point cloud is
 * not actually displayed
 */
```

bool

removePointCloud (const std::string &id = "cloud", int viewport = 0);

```
/** \brief Removes an added shape from screen (line, polygon, etc.), based on a given ID
 * \note This methods also removes PolygonMesh objects and PointClouds, if they match the ID
 * \param[in] id the shape object id (i.e., given on \a addLine etc.)
 * \param[in] viewport view port from where the Point Cloud should be removed (default: all)
 */
```

bool

removeShape (const std::string &id = "cloud", int viewport = 0);

```
/** \brief Remove all 3D shape data on screen from the given viewport.
```

```
 * \param[in] viewport view port from where the shapes should be removed (default: all)
```

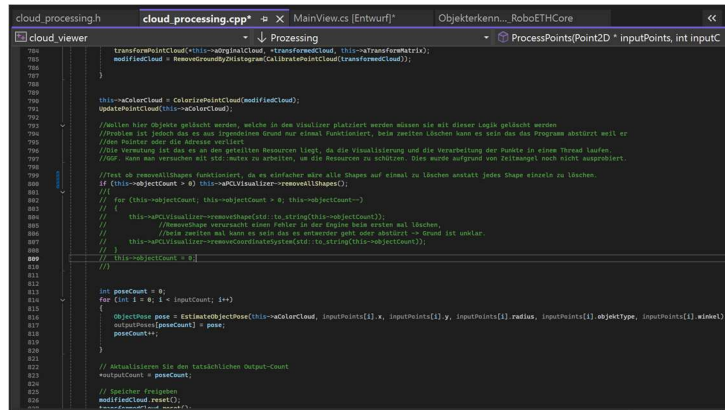
```
*/
```

```
bool
```

```
removeAllShapes (int viewport = 0);
```

//Test ob removeAllShapes funktioniert, da es einfacher wäre alle Shapes auf einmal zu löschen
anstatt jedes Shape einzeln zu löschen.

```
if (this->objectCount > 0) this->aPCLVisualizer->removeAllShapes();
```



13.05.26 | 11:26Uhr

NEUE VERSUCHE:

Änderungen am Code:

MainView.cs

```
private void tOfKameraVerbindenToolStripMenuItem_Click(object sender, EventArgs e)
{
```

```
    try
```

```
    {
```

```
        string input = Interaction.InputBox("Mit welcher Kamera möchten Sie sich  
verbinden?\n1=SICK 2=Schmersal");
```

```
        int inputInt = Convert.ToInt32(input);
```

```
        if (inputInt != 1 && inputInt != 2)
```

```
        {
```

```
            MessageBox.Show("Bitte geben Sie 1 oder 2 ein");
```

```
            return;
```

```
        }
```

```
        bool connected = this.derController.SetTofKamera(inputInt);
```

```
        // Luis Thiel - 20.05.2026
```

```
        DialogResult result = MessageBox.Show(
```

```
            "Möchten Sie die Ansicht in zwei Viewports (Original und Bearbeitet) aufteilen?",
```

```
            "Ansicht konfigurieren",
```

```
            MessageBoxButtons.YesNo,
```

```
            MessageBoxIcon.Question);
```

```
        // Auswerten, ob der Nutzer "Ja" geklickt hat
```

```
        bool useTwoViewports = (result == DialogResult.Yes);
```

```
        // Die Entscheidung an den Controller weitergeben
```

```
this.derController.SetViewportMode(useTwoViewports);
// -----
```

```
if (connected)
{
    MessageBox.Show("Erfolgreich mit der Kamera verbunden");
    tOFKameraVerbindenToolStripMenuItem.Checked = true;
    btnStartTof.Visible
}
```

Hier erfolgt das Erstellen einer Message Box, in welcher die Anzahl der Viewports abgefragt wird. Die Rückgabe ist ein boolean- Wert.

Mit **this.derController.SetViewportMode(useTwoViewports);** wird die Entscheidung an den Controller weitergegeben.

Controller.cs

Verfügt über folgende Funktion:

```
public void SetViewportMode(bool useTwo)
{
    this.aPointCloudProcessing.SetViewportMode(useTwo);
}
```

Hier wird die Entscheidung über das Attribut? useTwo an aPointCloudProcessing weitergegeben.

PointCloudProcessing.cs

Hier wird eine private Variable angelegt, um die Entscheidung zwischenspeichern bis die Parameterinitialisierung erfolgt.

```
private bool _useTwoViewports;
public void SetViewportMode(bool useTwoPorts)
{
    this._useTwoViewports = useTwoPorts;
}
```

Nun kann der Parameter twoViewports initialisiert werden:

```
private void InitParameters(int auswahlKamera)
{
    this.aObjektePose = new List<PoseAttributes>();
    this.aParameter = new Parameters();
    this.aParameter.selectedCamera = auswahlKamera;
    this.aParameter.calibrationPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
"calibrationData.json");

    this.aParameter.windowWidth = 1863;
    this.aParameter.windowHeight = 600;
    this.aParameter.textScale = 4;
}
```

```
if (auswahlKamera == 1)
{
    this.aParameter.twoViewports = this._useTwoViewports;
    this.aParameter.dataPath =
PathManagment.GetRelativePath(Path.Combine(PathManagment.GetPath(3, "Data"),
"data.ply"));
    this.aParameter.maxDistanceMeasure = -4;
    this.aParameter.minDistanceMeasure = -40;
```

Neuer Code = Rot